

Réseaux de neurones multi-couches

De la régression logistique à la rétropropagation

- 1 Linéaire contre non linéaire
- 2 Couche cachée
- 3 Propagation avant
- 4 Exemple : propagation avant

- 5 Perte et optimisation
- 6 Exemple : pas de gradient
- 7 Récapitulatif

Neurone de la classification linéaire

Le point de départ est la régression logistique : un produit scalaire, puis une sigmoïde qui écrase le score en probabilité.

$$\hat{y} = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

- Convention : x est augmenté d'une constante $x_0 = 1$, si bien que le poids w_0 **est** le biais.
- L'entraînement se fait par **descente de gradient**, $w \leftarrow w - \alpha \nabla_w L$, faute de forme close.
- La perte est l'**entropie croisée**, choisie en paire avec la sigmoïde.

Ces trois faits suffisent pour construire un réseau.

Un neurone ne trace qu'une droite

Les classifieurs linéaires (moindres carrés, perceptron, régression logistique) ajustent tous le **même** objet : un hyperplan $w^T x = 0$.

La limite est structurelle

Ce n'est pas une question d'algorithme ni de données d'entraînement. Un modèle linéaire *ne peut pas* produire une frontière courbe. Aucun réglage de α ou de λ n'y changera rien.

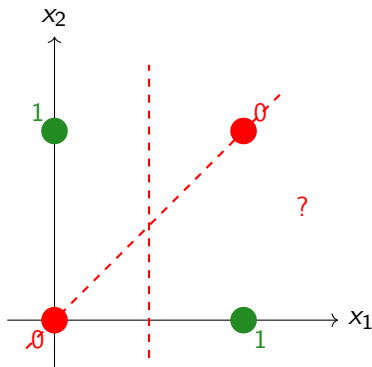
Il existe des problèmes minuscules, à deux entrées binaires, qu'aucun classifieur linéaire ne résout. Le plus célèbre tient en quatre points.

Quatre points qu'aucune droite ne sépare

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Règle : la sortie vaut 1 si **exactement une** entrée vaut 1.

Les deux classes occupent les **diagonales** du carré. Aucune droite ne peut les séparer.



D'où vient la non-linéarité ?

Aucun modèle linéaire ne résout XOR sur (x_1, x_2) . Il y parvient dès qu'on **transforme les entrées**.

Transformation choisie à la main

Ajouter une fonction de base, le produit $x_1 x_2$:

$$\text{XOR} = x_1 + x_2 - 2 x_1 x_2$$

Linéaire en $\phi(x) = (x_1, x_2, x_1 x_2)$, mais il a fallu deviner $x_1 x_2$.

Transformation apprise

Un réseau de neurones transforme lui aussi les entrées, mais personne ne choisit la transformation : elle est **apprise des données**, en même temps que le classifieur.

Un modèle, une perte, une optimisation

Tout modèle supervisé se décrit par ces trois pièces. Voici comment elles se remplissent pour un réseau.

Le modèle	des couches d'unités, exécutées par la <i>propagation avant</i>
La perte	adaptée à la tâche par la dernière couche
L'optimisation	la descente de gradient, propulsée par la <i>rétropropagation</i>

Les deux premières lignes sont des assemblages de pièces déjà connues. La troisième contient la seule vraie nouveauté : la rétropropagation.

Plan du cours

- 1 Linéaire contre non linéaire
- 2 **Couche cachée**
- 3 Propagation avant
- 4 Exemple : propagation avant
- 5 Perte et optimisation
- 6 Exemple : pas de gradient
- 7 Récapitulatif

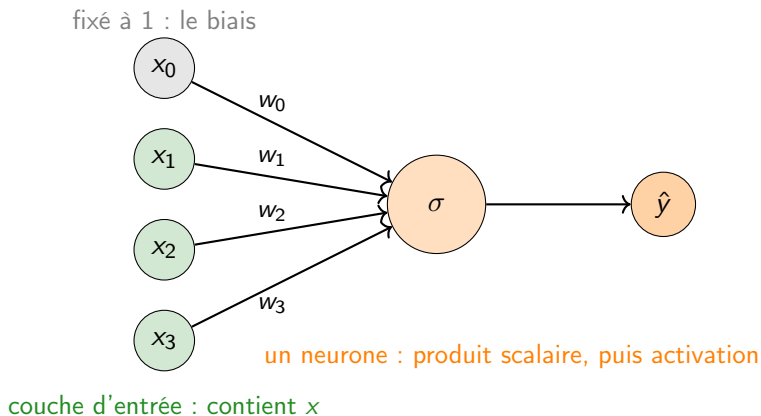
La régression logistique est déjà un réseau

Un produit scalaire avec les poids w , puis une sigmoïde. Dessiné comme un graphe, c'est déjà un réseau, le plus petit possible.

$$\hat{y} = \sigma(w^T x)$$

- La **couche d'entrée** contient x et ne calcule rien.
- Un seul **neurone de sortie** fait tout le travail : multiplier par les poids, appliquer l'activation.
- Chaque neurone d'un réseau est exactement cette unité.

Réseau minimal en image



Chaque arête porte un poids, et le neurone applique σ à la somme pondérée. Le neurone x_0 , fixé à 1, porte le biais : son poids est w_0 .

Insérer une couche cachée

Rien n'oblige le neurone de sortie à lire l'entrée **brute**. Insérons trois neurones entre l'entrée et la sortie. Chacun est la même unité, avec ses propres poids :

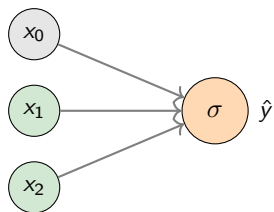
$$a_i = g(w_i^T x), \quad i = 1, 2, 3 \quad \implies \quad a = g(W^{[1]}x)$$

en empilant les w_i^T comme **lignes** de $W^{[1]}$. Le neurone de sortie, lui, n'a pas changé du tout :

$$\hat{y} = \sigma(w^{[2T]}a) = \sigma(w^{[2T]}g(W^{[1]}x))$$

- C'est toujours la régression logistique : elle lit simplement les trois valeurs apprises a , augmentées de $a_0 = 1$, au lieu de x .
- L'exposant $[l]$ dit à quelle **couche** un symbole appartient. Le modèle a maintenant deux jeux de poids : il faut bien les nommer.

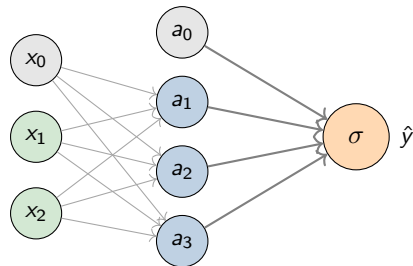
Insertion en image



la régression logistique

$$\hat{y} = \sigma(w^T x)$$

$\Rightarrow$



$W^{[1]}$

le neurone de sortie est inchangé

Le neurone de sortie orange est identique dans les deux dessins. Rendre le modèle profond a changé ce qu'il lit : trois caractéristiques apprises a au lieu du x brut. Chaque couche porte son propre neurone constant.

La non-linéarité est obligatoire

Si g était l'identité, les deux couches se réduiraient à une seule application linéaire :

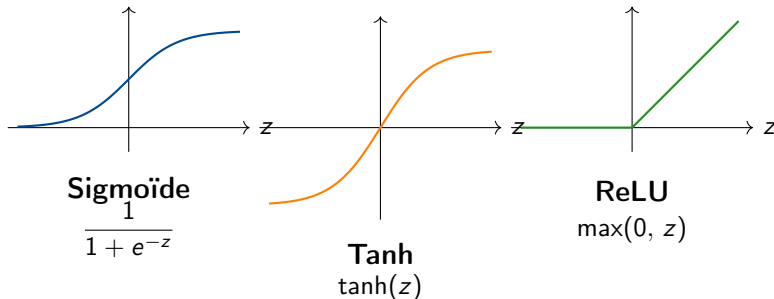
$$W^{[2]}(W^{[1]}x) = (W^{[2]}W^{[1]})x = W'x$$

- Le produit de deux matrices est une matrice. La profondeur n'apporterait strictement rien : on retomberait sur un unique classifieur linéaire.
- Cela vaut pour n'importe quelle profondeur : cent couches linéaires empilées restent une seule application linéaire.

C'est g qui fait tout

La non-linéarité entre les couches est ce qui rend l'empilement utile, et c'est elle qui met XOR à portée.

Quelles activations ?



La sigmoïde suffit

Le choix de g a des conséquences lourdes (saturation, gradients qui disparaissent, vitesse d'entraînement). Ici, seule compte la propriété d'être *non linéaire*.

La couche cachée apprend les caractéristiques

- Le neurone de sortie reste un **classifieur linéaire** : il n'a pas changé d'une ligne.
- Le travail de la couche cachée est donc de **déplacer les données** là où les classes deviennent linéairement séparables.
- C'est exactement le rôle des fonctions de base ϕ , avec une amélioration décisive : ϕ était fixée à la main *avant* l'entraînement, tandis que a est **apprise des données**, en même temps que le reste.

Division du travail

La couche cachée apprend une représentation. La couche de sortie y trace un hyperplan.

XOR résolu par une couche cachée

Deux unités cachées suffisent, avec un seuil dur pour lire le résultat facilement :

$$W^{[1]} = \begin{pmatrix} -0,5 & 1 & 1 \\ -1,5 & 1 & 1 \end{pmatrix}, \quad w^{[2]} = (-0,5 \quad 1 \quad -1)$$

x_1	x_2	a_1	a_2	$\hat{y}$
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

- a_1 : au moins une entrée à 1, un **OU**.
- a_2 : les deux à 1, un **ET**.
- La sortie lit $a_1 - a_2$.

Explication

Dans le plan (a_1, a_2) , les quatre points sont devenus linéairement séparables. La couche cachée n'a pas classé : elle a **changé de repère**.

Plan du cours

- 1 Linéaire contre non linéaire
- 2 Couche cachée
- 3 Propagation avant**
- 4 Exemple : propagation avant
- 5 Perte et optimisation
- 6 Exemple : pas de gradient
- 7 Récapitulatif

Empiler les couches

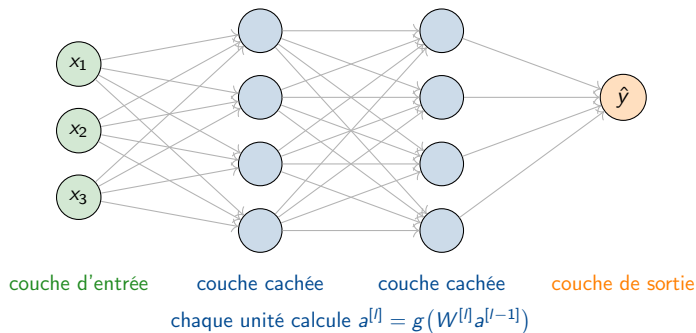
Une couche cachée a fonctionné, alors répétons le geste : le vecteur $a^{[1]}$ peut alimenter une deuxième couche cachée, dont la sortie $a^{[2]}$ peut en alimenter une troisième, jusqu'à une couche de sortie qui produit $\hat{y}$.

- La couche l possède sa matrice $W^{[l]}$ (**une ligne par neurone**) et son activation $g^{[l]}$.
- La **largeur** (unités par couche) et la **profondeur** (nombre de couches) sont les deux boutons de capacité.

Prix de la capacité

Plus d'unités et plus de couches, c'est plus de paramètres et des frontières plus expressives, et plus de place pour **surapprendre**.

Entrée, couches cachées, sortie



Chaque arête porte un poids de $W^{[l]}$. Les neurones de biais constants ne sont pas dessinés ici.

Formule générale

Calculer la prédiction en lisant le réseau de gauche à droite s'appelle la **propagation avant**. La formule ne fait que redire, une fois par couche, ce qu'on vient de construire :

$$z^{[l]} = W^{[l]} a^{[l-1]}, \quad a^{[l]} = g^{[l]}(z^{[l]}), \quad a^{[0]} = x, \quad \hat{y} = a^{[L]}$$

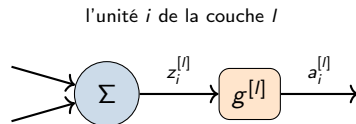
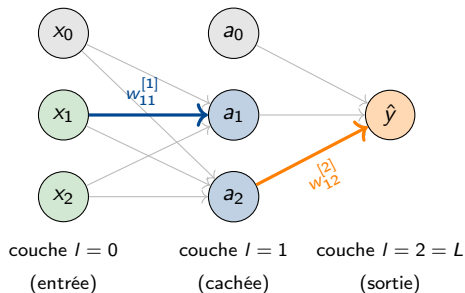
- $z^{[l]}$ est la **pré-activation** : la somme pondérée, avant g .
- Convention à retenir : chaque $a^{[l]}$, comme l'entrée, se lit avec son neurone de biais $a_0^{[l]} = 1$ ajouté en tête.
- Deux lignes de calcul par couche, répétées L fois. C'est tout le modèle.

Sous forme vectorisée, tout le **mini-lot** circule d'un coup : une opération matricielle par couche, avec les exemples en colonnes.

$$Z^{[l]} = W^{[l]}A^{[l-1]}$$

- $A^{[l-1]}$ empile les activations de tous les exemples du lot, une colonne chacun.
- À la fois plus clair à écrire et bien plus rapide à exécuter : c'est cette forme qui tourne sur GPU.

Formule placée sur le graphe



à l'intérieur d'une unité :
somme pondérée, puis activation

*L'exposant $[l]$ nomme la couche. Un poids $w_{ij}^{[l]}$ va **vers** l'unité i de la couche l , **depuis** l'unité j de la couche $l - 1$, les indices se lisent de droite à gauche, comme dans un produit matriciel.*

Nommer les arêtes et les poids

Symbole	Nom	Où il vit sur le graphe
l, L	indice de couche, nombre de couches	la colonne d'unités, $l = 0$ est l'entrée
$x = a^{[0]}$	l'entrée	la colonne la plus à gauche
$w_{ij}^{[l]}$	un poids	l'arête vers l'unité i de la couche l , depuis l'unité j
$W^{[l]}$	la matrice de poids	les arêtes arrivant dans la couche l , une ligne par unité
$x_0, a_0^{[l]}$	le neurone de biais	une unité fixée à 1 ; $w_{i0}^{[l]}$ est le biais

À l'intérieur d'une unité

Symbole	Nom	Où il vit sur le graphe
$z_i^{[l]}$	la pré-activation	la somme pondérée, calculée avant g
$g^{[l]}$	la fonction d'activation	appliquée dans chaque unité de la couche l
$a_i^{[l]}$	l'activation	la valeur envoyée sur les arêtes sortantes
$\hat{y} = a^{[L]}$	la prédiction	ce qui sort de la dernière couche

Plan du cours

- 1 Linéaire contre non linéaire
- 2 Couche cachée
- 3 Propagation avant
- 4 Exemple : propagation avant**
- 5 Perte et optimisation
- 6 Exemple : pas de gradient
- 7 Récapitulatif

Faisons tourner ce réseau exact avec des nombres. L'entrée est $x = (1, 2)$, augmentée en $(1, 1, 2)$ par le neurone de biais, et la sigmoïde sert d'activation partout :

$$W^{[1]} = \begin{pmatrix} 0 & 2 & -1 \\ -1 & 1 & 1 \end{pmatrix}, \quad W^{[2]} = \begin{pmatrix} 0 & 1 & 1 \end{pmatrix}$$

- La **ligne** i de $W^{[1]}$ rassemble les poids des arêtes qui arrivent dans l'unité cachée i , **biais en tête**.
- $W^{[2]}$ n'a qu'une ligne : la couche de sortie n'a qu'une unité.

Couche cachée unité par unité

$$z_1^{[1]} = \underbrace{0}_{w_{10}^{[1]}} \cdot \underbrace{1}_{x_0} + \underbrace{2}_{w_{11}^{[1]}} \cdot \underbrace{1}_{x_1} + \underbrace{(-1)}_{w_{12}^{[1]}} \cdot \underbrace{2}_{x_2} = 0, \quad a_1^{[1]} = \sigma(0) = 0,5$$

$$z_2^{[1]} = -1 \cdot 1 + 1 \cdot 1 + 1 \cdot 2 = 2, \quad a_2^{[1]} = \sigma(2) \approx 0,88$$

- L'unité 1 tombe exactement sur zéro, le point milieu de la sigmoïde : elle sort 0,5, c'est-à-dire « aucun avis ».
- L'unité 2 se place haut sur la courbe.

La couche de sortie répète les deux mêmes étapes. Elle lit cette fois $a^{[1]} = (0,5, 0,88)$, augmenté en $(1, 0,5, 0,88)$ par son **propre** neurone de biais, au lieu de x :

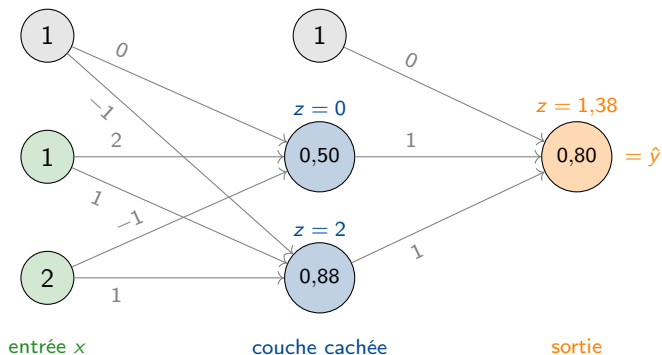
$$z^{[2]} = 0 \cdot 1 + 1 \cdot 0,5 + 1 \cdot 0,88 = 1,38, \quad \hat{y} = a^{[2]} = \sigma(1,38) \approx 0,80$$

Le réseau prédit la classe 1 avec une probabilité d'environ 0,80.

C'est tout ce que fait la propagation avant

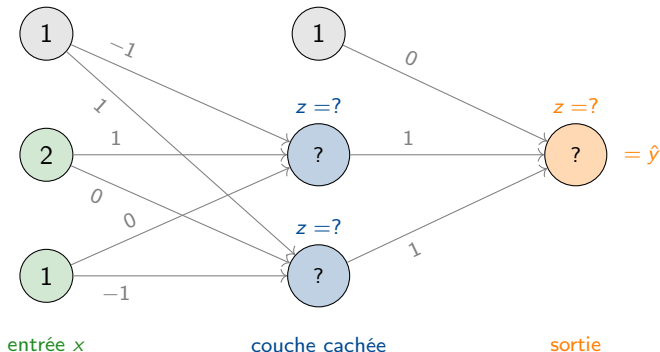
Multiplier par les poids des arêtes, neurone de biais compris, appliquer l'activation. À chaque unité, de chaque couche.

Calcul sur le dessin



Chaque arête porte son poids, chaque unité affiche sa pré-activation z puis son activation a . Suivre le dessin de gauche à droite, c'est refaire le calcul.

Exercice : à vous de propager



Lire $W^{[1]}$ et $w^{[2]}$ sur le dessin (biais en tête), calculer $z^{[1]}$ puis $a^{[1]}$, puis $z^{[2]}$ et $\hat{y}$, et dire quelle classe le réseau prédit.

Aide : $\sigma(-1) \approx 0,27$ $\sigma(0) = 0,50$ $\sigma(1) \approx 0,73$ $\sigma(1,2) \approx 0,77$ $\sigma(2) \approx 0,88$.

Exercice : la correction

Couche cachée

$$z_1^{[1]} = -1 \cdot 1 + 1 \cdot 2 + 0 \cdot 1 = 1, \text{ donc } a_1^{[1]} = \sigma(1) \approx 0,73.$$

$$z_2^{[1]} = 1 \cdot 1 + 0 \cdot 2 + (-1) \cdot 1 = 0, \text{ donc } a_2^{[1]} = \sigma(0) = 0,50.$$

Sortie

En augmentant $a^{[1]}$ de son neurone de biais, $(1, 0,73, 0,50)$:

$$z^{[2]} = 0 \cdot 1 + 1 \cdot 0,73 + 1 \cdot 0,50 = 1,23, \quad \hat{y} = \sigma(1,23) \approx 0,77$$

Le réseau prédit la **classe 1**, avec une probabilité d'environ 0,77.

Plan du cours

- 1 Linéaire contre non linéaire
- 2 Couche cachée
- 3 Propagation avant
- 4 Exemple : propagation avant
- 5 Perte et optimisation**
- 6 Exemple : pas de gradient
- 7 Récapitulatif

La tâche vit dans la dernière couche

Le **corps** du réseau ignore complètement la tâche : ce sont les mêmes unités, qu'on fasse de la régression ou de la classification à mille classes.

La tâche vit dans la **dernière couche** : son activation met en forme $\hat{y}$, et la perte compare $\hat{y}$ à l'étiquette.

Tâche	Activation de sortie	Fonction de perte
régression	identité	erreur quadratique
classification binaire	sigmoïde	entropie croisée binaire
classification multiclasse	softmax	entropie croisée catégorielle

Ces trois lignes sont exactement les pertes des modèles linéaires.

Écrites sur la pré-activation z de la dernière couche :

$$\hat{y} = \frac{1}{1 + e^{-z}} \quad (\text{binaire}) \qquad \hat{y}_c = \frac{e^{z_c}}{\sum_j e^{z_j}} \quad (\text{multiclasse})$$

Perte et activation se choisissent en paire

Erreur classique

Choisir la perte et l'activation de sortie **indépendamment**. Elles vont par deux, et les dépareiller casse l'entraînement.

- L'entropie croisée appelle une **softmax** (ou une sigmoïde) : c'est l'appariement qui fait disparaître le facteur $\hat{y}(1 - \hat{y})$ du gradient.
- L'erreur quadratique appelle une sortie **identité** : borner la sortie n'aurait aucun sens quand la cible est un réel quelconque.
- Une sigmoïde entraînée à l'erreur quadratique sature et n'apprend presque plus.

Étape 0 : poser l'objectif

Entraîner un réseau suit les mêmes étapes que tout modèle supervisé. D'abord l'objectif : les poids qui minimisent la perte, plus un régulariseur qui les garde petits.

$$W^* = \arg \min_W L(W) + \lambda R(W), \quad R(W) = \|W\|_1 \text{ ou } \|W\|_2^2$$

- C'est la recette du **maximum a posteriori** de la régression linéaire.
- Le régulariseur L_2 s'appelle ici *weight decay*.

Étape 2 : descendre le gradient

Mettre à jour chaque poids d'un petit pas contre son gradient, avec le taux d'apprentissage α :

$$w_{ij}^{[l]} \leftarrow w_{ij}^{[l]} - \alpha \frac{\partial (L + \lambda R)}{\partial w_{ij}^{[l]}}$$

- Exactement la descente de gradient de la régression logistique, appliquée à *tous* les poids de *toutes* les couches, simultanément.
- Rien de nouveau dans la règle. Toute la difficulté est de calculer ce gradient quand $w_{ij}^{[l]}$ est enfoui au milieu de la pile.

Le gradient se calcule sur un mini-lot

La classification linéaire offrait deux extrêmes : le **lot complet** (précis, lent) ou un **seul exemple** (rapide, bruité). Les réseaux s'entraînent entre les deux.

Compromis du mini-lot

Un gradient assez précis pour progresser, un pas assez bon marché pour en faire des milliers.

- La forme vectorisée $Z^{[l]} = W^{[l]}A^{[l-1]}$ traite tout le mini-lot en **un** produit matriciel par couche.
- La taille du lot est un hyperparamètre, au même titre que α et λ .

Étape 3 : la rétropropagation

La pièce vraiment nouvelle : calculer $\partial L / \partial w_{ij}^{[l]}$ pour chaque poids d'une pile de couches. La rétropropagation y parvient par la **règle de dérivation en chaîne**, en une passe avant et une passe arrière.

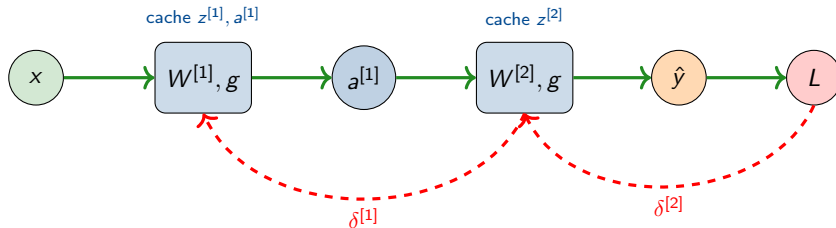
Avec l'**erreur de couche** $\delta^{[l]} = \partial L / \partial z^{[l]}$:

$$\delta^{[l]} = \left(\left(W^{[l+1]} \right)^T \delta^{[l+1]} \right) \odot g'^{[l]} \left(z^{[l]} \right), \quad \frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} \left(a^{[l-1]} \right)^T$$

- $\odot$ est le produit **terme à terme** : $(u \odot v)_i = u_i v_i$.
- La passe avant **met en cache** chaque $z^{[l]}$ et $a^{[l]}$; la passe arrière les réutilise.

Deux passes en image

passage avant : calculer et mettre en cache



passage arrière : propager l'erreur en chaîne

La propagation avant calcule et met en cache les activations. La rétropropagation renvoie le gradient de la perte par les mêmes arêtes, en sens inverse.

Plan du cours

- 1 Linéaire contre non linéaire
- 2 Couche cachée
- 3 Propagation avant
- 4 Exemple : propagation avant
- 5 Perte et optimisation
- 6 Exemple : pas de gradient**
- 7 Récapitulatif

Reprenons le réseau exactement où la propagation avant l'a laissé :

$$x = (1, 1, 2), \quad a^{[1]} = (1, 0,5, 0,88), \quad \hat{y} = 0,80$$

Donnons maintenant à l'exemple sa vraie classe : $y = 0$. Avec l'entropie croisée binaire et sans régularisation ($\lambda = 0$), la perte vaut

$$L = -\ln(1 - \hat{y}) = -\ln(0,20) \approx 1,61$$

Le réseau se trompe avec assurance

Il annonce 0,80 pour la classe 1 alors que la vraie classe est 0. Le gradient va le lui dire.

Erreur de sortie

Pour une sortie sigmoïde entraînée avec l'entropie croisée, $\delta^{[2]}$ se réduit au familier résidu de la classification linéaire :

$$\delta^{[2]} = \hat{y} - y = 0,80 - 0 = 0,80$$

Chaque poids de $W^{[2]}$ reçoit $\delta^{[2]}$ fois l'activation qu'il lit, la formule du produit extérieur, neurone de biais compris :

$$\frac{\partial L}{\partial W^{[2]}} = \delta^{[2]} \left(a^{[1]} \right)^T = 0,80 \cdot (1, 0,5, 0,88) = (0,80, 0,40, 0,70)$$

Le poids du biais prend la plus grosse part : sa « activation » vaut 1, la plus grande des trois.

En arrière à travers la couche cachée

Chaque unité cachée prend sa part de l'erreur à **travers son poids sortant**, fois sa propre pente $\sigma'(z) = \sigma(z) (1 - \sigma(z))$:

$$\delta_1^{[1]} = \underbrace{1}_{w_{11}^{[2]}} \cdot \underbrace{0,80}_{\delta^{[2]}} \cdot \underbrace{0,25}_{\sigma'(0)} = 0,20, \quad \delta_2^{[1]} = 1 \cdot 0,80 \cdot 0,10 = 0,08$$

Puis le même produit extérieur, cette fois contre $x = (1, 1, 2)$:

$$\frac{\partial L}{\partial W^{[1]}} = \delta^{[1]} x^T = \begin{pmatrix} 0,20 & 0,20 & 0,40 \\ 0,08 & 0,08 & 0,16 \end{pmatrix}$$

- Le neurone de biais constant ne prend **aucune** erreur : il n'a pas de poids entrant.
- Notez la petite pente 0,10 de l'unité 2, haut sur la sigmoïde : l'erreur qui la traverse est divisée par dix.

L'étape 2 avec un $\alpha = 1$ volontairement grand, pour rendre le mouvement visible :

$$W^{[2]} \leftarrow (0, 1, 1) - (0,80, 0,40, 0,70) = (-0,80, 0,60, 0,30)$$

$$W^{[1]} \leftarrow \begin{pmatrix} 0 & 2 & -1 \\ -1 & 1 & 1 \end{pmatrix} - \begin{pmatrix} 0,20 & 0,20 & 0,40 \\ 0,08 & 0,08 & 0,16 \end{pmatrix} = \begin{pmatrix} -0,20 & 1,80 & -1,40 \\ -1,08 & 0,92 & 0,84 \end{pmatrix}$$

Toutes les couches sont mises à jour **en même temps**, à partir d'une seule passe arrière.

Est-ce que ça a aidé ?

Relançons la propagation avant avec les nouveaux poids :

$$z^{[1]} = (-1,20, 1,52), \quad a^{[1]} = (0,23, 0,82), \quad z^{[2]} = -0,42$$

$$\hat{y} = \sigma(-0,42) \approx 0,40, \quad L = -\ln(1 - 0,40) \approx 0,51$$

Un seul pas

La prédiction de la classe 1 est passée de 0,80 à 0,40, et la perte de 1,61 à 0,51. ■

L'entraînement, c'est cette boucle (propagation avant, rétropropagation, mise à jour) répétée sur les mini-lots jusqu'à ce que la perte se stabilise.

Plan du cours

- 1 Linéaire contre non linéaire
- 2 Couche cachée
- 3 Propagation avant
- 4 Exemple : propagation avant
- 5 Perte et optimisation
- 6 Exemple : pas de gradient
- 7 Récapitulatif**

Modèle	des couches d'unités « produit scalaire puis activation »
Propagation avant	$z^{[l]} = W^{[l]}a^{[l-1]}$, $a^{[l]} = g^{[l]}(z^{[l]})$, prédiction $\hat{y} = a^{[L]}$
Caractéristiques	appprises par les couches cachées ; la sortie reste un classifieur linéaire
Perte	en paire avec la tête : identité/quadratique, sigmoïde ou softmax/entropie croisée
Optimisation	$W^* = \arg \min_W L + \lambda R$, descente de gradient par mini-lots
Gradients	par rétropropagation : $\delta^{[l]}$ redescend le câblage, un produit extérieur par couche
Capacité	largeur et profondeur, réglées avec α , λ et la taille du mini-lot
